

Defer Sender Algorithm Customization to C++29

Document #:	P3826R0 [Latest] [Status]
Date:	2025-10-05
Project:	Programming Language C++
Audience:	SG1 Concurrency and Parallelism Working Group LEWG Library Evolution Working Group LWG Library Working Group
Reply-to:	Eric Niebler < eric.niebler@gmail.com >

Contents

- [1 Background](#)
- [2 The problem with P3718](#)
- [3 Solutions considered](#)
 - [3.1 Remove all of the C++26 `std::execution` additions](#)
 - [3.2 Remove all of the customizable sender algorithms](#)
 - [3.3 Remove sender algorithm customization](#)
 - [3.4 Ship everything as-is and fix algorithm customization in a DR](#)
- [4 Implications of removal](#)
 - [4.1 The parallel scheduler](#)
 - [4.2 The task scheduler](#)
 - [4.3 The bulk algorithms](#)
 - [4.3.1 Option 1: Remove bulk, bulk_chunked, and bulk_unchunked](#)
 - [4.3.2 Option 2: Magical parallel execution](#)
 - [4.3.3 Option 3: A normative mechanism for the bulk* algorithms only](#)
 - [4.4 Impacts on hardware vendors](#)
- [5 Mitigating factors](#)
 - [5.1 Sender introspection](#)
 - [5.2 Third party algorithms](#)
 - [5.3 Difficulties with proprietary extensions](#)

- 6 The removal process
 - 6.1 Approach
 - 6.2 Procedure
- 7 Restoring algorithm customization in C++29
 - 7.1 Completion scheduler enhancements
 - 7.2 Domains
 - 7.3 Customizing connect
 - 7.4 The parallel and task schedulers and bulk
 - 7.5 `inline_scheduler`
- 8 Proposed wording
- 9 Appendix A: The planned fix
 - 9.1 Mission statement
 - 9.2 Achieving the mission
 - 9.2.1 Coloring the graph
 - 9.2.2 Dispatching in connect
- 10 References

§ 1 Background

In the current Working Draft, 33 [\[exec\]](#) has sender algorithms that are customizable. While the sender/receiver concepts and the algorithms themselves have been stable for several years now, the customization mechanism has seen a fair bit of recent churn. [\[P3718R0\]](#) is the latest effort to shore up the mechanism. Unfortunately, there are gaps in its proposed resolution. This paper details those gaps.

The problems are fixable although the fixes are non-trivial. The time for elaborate fixes has passed. This paper proposes to remove the ability to customize sender algorithms for C++26. A future paper will propose to add the feature back post-'26.

The author feels that postponing the feature will be less disruptive and safer than trying to patch it at the last minute. Most common usages of sender/receiver will not be affected.

§ 2 The problem with P3718

[\[P3718R0\]](#) identifies real problems with the status quo of sender algorithm customization. It proposes using information from the sender about where it will *complete* during “early” customization, which happens when a sender algorithm constructs and returns a sender; and it proposes using information from the receiver about where the operation will *start* during “late” customization, when the sender and the receiver are connected.

The problem with this separation of responsibilities is that many senders do not know where they will complete until they know where they will be started. A simple example is the `just()` sender; it completes inline wherever it is started. And the information about where a sender will start is not known during early customization, when the sender is being asked for this information.

For the expression `then(sndr, fn)` for example, if the then CPO asks `sndr` where it will complete, `sndr` might not be able to answer, in which case no “early” customization is performed. And during “late” (connect-time) customization, only the receiver’s information about where the operation will start is used to find a customization. Presumably an algorithm like `then(sndr, fn)` would want to dispatch based on where the function `fn` will execute, but for some expressions that information cannot be determined with the API proposed in P3718.

An illustrative example is:

```
namespace ex = std::execution;
auto sndr = ex::starts_on(gpu, ex::just()) | ex::then(fn);
std::this_thread::sync_wait(std::move(sndr));
```

... where `gpu` is a scheduler that runs work (unsurprisingly) on a GPU.

`fn` will execute on the GPU, so a GPU implementation of `then` should be used. By the proposed resolution of P3718, algorithm customization proceeds as follows:

- During early customization, when `starts_on(gpu, just()) | then(fn)` is executing, the then CPO asks the `starts_on(gpu, just())` sender where it will complete as if by:

```
auto&& tmp1 = ex::starts_on(gpu, ex::just());
auto dom1 = ex::get_domain(ex::get_env(tmp1));
```

- The `starts_on` sender will in turn ask the `just()` sender, as if by:

```
auto&& tmp2 = ex::just();
auto dom2 = ex::get_domain(ex::get_env(tmp2));
```

As discussed, the `just()` sender doesn’t know where it will complete until it knows where it will be started, but that information is not yet available. As a result, `dom2` ends up as `default_domain`, which is then reported as the domain for the `starts_on` sender. That’s incorrect. The `starts_on` sender will complete on the GPU.

- The then CPO uses `default_domain` to find an implementation of the then algorithm, which will find the default implementation. As a result, the then CPO returns an ordinary then sender.
- When that then sender is connected to `sync_wait`’s receiver, late customization happens. `connect` asks `sync_wait`’s receiver where the then sender will be started. It does that with `get_domain(get_env(rcvr))`. `sync_wait` starts operations on the cur-

rent thread, so the `get_domain` query will return `default_domain`. As with early customization, late customization will also not find a GPU implementation.

The end result of all of this is that a default (which is effectively a CPU) implementation will be used to evaluate the then algorithm on the GPU. That is a bad state of affairs.

§ 3 Solutions considered

OK, so there is a problem. What do we do? There are a number of different options.

§ 3.1 Remove all of the C++26 `std::execution` additions

Although the safest option, I hope most agree that such a drastic step is not warranted by this issue. Pulling the sender abstraction and everything that depends on it would result in the removal of:

- The sender/receiver-related concepts and customization points, without which the ecosystem will have no shared async abstraction, and which will set back the adoption of structured concurrency three years.
- The sender algorithms, which capture common async patterns and make them reusable,
- `execution::counting_scope` and `execution::simple_counting_scope`, and related features for incremental adoption of structured concurrency,
- `execution::parallel_scheduler` and all of its related APIs, and
- `execution::task` and `execution::task_scheduler` (C++26 will *still* not have a standard coroutine task type *<heavy sigh>*).

This option should only be considered if all the other options are determined to have unacceptable risk.

§ 3.2 Remove all of the customizable sender algorithms

This option would keep all of the above library components with the exception of the customizable sender algorithms:

- `then`, `upon_error`, `upon_stopped`
- `let_value`, `let_error`, `let_stopped`
- `bulk`, `bulk_chunked`, `bulk_unchunked`
- `starts_on`, `continues_on`, `on`
- `when_all`, `when_all_with_variant`
- `stopped_as_optional`, `stopped_as_error`

- `into_variant`
- `sync_wait`
- `affine_on`

This would leave users with no easy standard way to start work on a given execution context, or transition to another execution context, or to execute work in parallel, or to wait for work to finish.

In fact, without the bulk algorithms, we leave no way for the `parallel_scheduler` to execute work in parallel!

While still delivering a standard async abstraction with minimal risk, the loss of the algorithms would make it *just* an abstraction. Like coroutines, adoption of senders as an async *lingua franca* will be hampered by lack of standard library support.

§ 3.3 Remove sender algorithm customization

This is the option this paper proposes. We ship everything currently in the Working Draft but remove the ability to customize the algorithms. This gives us a free hand to design a better customization mechanism for C++29 – provided we have high confidence that those new customization hooks can be added without break existing behavior.

A fair question is: how can we have such certainty when we do not know what the customization hooks are yet?

To answer that question for myself, I implemented new customization hooks [here](#) that address the known issues. Using that design (described in [Appendix A: The planned fix](#)) as a polestar, this paper proposes wording to remove customization in such a way that will let us add it back later without breakage.

My experience implementing the solution gives me confidence that we can introduce that solution or one like it later without compatibility problems.

§ 3.4 Ship everything as-is and fix algorithm customization in a DR

This option is not as reckless as it sounds. I describe the shape of a possible fix in [Appendix A: The planned fix](#). It would not be the first time the Committee shipped a standard with known defects, and the DR process exists for just this purpose.

What gives me pause, however, is the fact that I have “fixed” this problem before only to find that my fix is broken, and not just once!

I have implemented my planned fix, and it *seems* to work, but it has not seen any real-world usage. In short, my confidence is not high enough to endorse this solution.

Should someone with sufficient interest come and vet my solution, I might change my mind. Shipping it as-is is certainly the least amount of work for everyone involved.

§ 4 Implications of removal

Removing algorithm customization is fairly straightforward in most regards, but there are a few parts of `std::execution` that need special care.

§ 4.1 The parallel scheduler

The `parallel_scheduler` goes to great lengths to ensure that the bulk family of algorithms – `bulk`, `bulk_chunked`, and `bulk_unchunked` – are executed in parallel when the users requests it and when the underlying execution context supports it.

To that end, the `parallel_scheduler` “provides a customized implementation” of the `bulk_chunked` and `bulk_unchunked` algorithms, but nothing is said about how those custom implementations are found or under what circumstances users can be assured that the `parallel_scheduler` will use them. Arguably, this is under-specified in the current Working Draft and should be addressed whether this paper is accepted or not.

We have to give users a guarantee that if X , Y , and Z conditions are met, `bulk[_[un]chunked]` will be run in parallel with absolute certainty.

One solution is to say that the bulk algorithms are guaranteed to execute in parallel when the immediate predecessor of the bulk operation is known to complete on the `parallel_scheduler`. In a sender expression such as the following:

```
sndr | std::execution::bulk(std::par, 1024, fn)
```

If `sndr`’s attributes advertizes a completion scheduler of type `parallel_scheduler`, then we can guarantee that the bulk operation will execute in parallel. Implementations can choose to parallelize bulk under other circumstances, but we require this one.

The implication of offering this guarantee is that we must preserve the guarantee going forward. Any new customization mechanism we might add must *never* result in parallel execution becoming serialized.

The reverse is not necessarily true though. I maintain that a future change that parallelizes a bulk algorithm that formerly executed serially on the `parallel_scheduler` is an acceptable change of behavior.

If SG1 or LEWG disagrees, there are ways to avoid even this behavior change.

§ 4.2 The task scheduler

Library issue [#4336](#) describes the poor interaction between `task_scheduler`, a type-erased scheduler, and the bulk family of algorithms; namely, that the `task_scheduler` al-

ways executes bulk in serial, even when it is wrapping a `parallel_scheduler`.

This is not a problem caused by the customization mechanism, but it is something that can be addressed as part of the customization removal process.

When we address that issue, we must avoid the `parallel_scheduler` pitfall by under-specifying the interaction with bulk. As with `parallel_scheduler`, users must have a guarantee about the conditions under which bulk is accelerated on a `task_scheduler`.

Fortunately, the `parallel_scheduler` has already given us a way to punch the `bulk_chunked` and `bulk_unchunked` algorithms through a type-erased API boundary: `parallel_scheduler_backend` (33.16.3 [[exec.sysctxrepl.psb](#)]). By specifying the behavior of `task_scheduler` in terms of `parallel_scheduler_backend` and `bulk_item_receiver_proxy`, we can give `task_scheduler` the ability to parallelize bulk without having to invent a new mechanism.

§ 4.3 The bulk algorithms

Few users will ever have a need to customize an algorithm like `then` or `let_value`. The bulk algorithms are a different story. Anybody with a custom thread pool will benefit from a custom bulk implementation that can run in parallel on the thread pool. The loss of algorithm customization is particularly painful in this area. This section explores some options to address these concerns and makes a recommendation.

§ 4.3.1 Option 1: Remove `bulk`, `bulk_chunked`, and `bulk_unchunked`

This option cuts the Gordian knot, but comes at a high cost. The `parallel_scheduler` can hardly be called “parallel” if it does not offer a way to execute work in parallel, so cutting the bulk algorithms probably means cutting `parallel_scheduler` also.

§ 4.3.2 Option 2: Magical parallel execution

In this option, we keep the bulk algorithms and the `parallel_scheduler`, and we say that the bulk algorithms are executed in parallel on the `parallel_scheduler` (and on a `task_scheduler` that wraps a `parallel_scheduler`), but we leave the mechanism unspecified.

This option is essentially the *status quo*, except that as discussed in [The parallel scheduler](#), this aspect of the `parallel_scheduler` is currently under-specified. The referenced section proposes a path forward.

A variant of this option is to specify an exposition-only mechanism whereby bulk gets parallelized.

This option makes `parallel_scheduler` and `task_scheduler` “magic” with respect to the bulk algorithms. End users would have no standard mechanism to parallelize bulk on their own third-party thread pools in C++26.

This is the approach taken by the [Proposed wording](#) below.

§ 4.3.3 Option 3: A normative mechanism for the bulk* algorithms only

In this option, we reintroduce algorithm customization with a special-purpose API just for the bulk algorithms. For example, a scheduler might have an optional `sch.bulk_transform(sndr, env)` that turns a serial bulk* sender into one that executes in parallel on scheduler `sch`. Whenever a bulk* sender is passed to `connect`, `connect` can check the sender’s predecessor for a completion scheduler that defines `bulk_transform` and uses it if found.

The downside of this approach is that we will still have to support this API even when a more general algorithm customization mechanism is available. That doesn’t seem terribly onerous to me, but that is for SG1/LEWG to decide.

§ 4.4 Impacts on hardware vendors

Without algorithm customization, manufacturers of special-purpose hardware accelerators will not be able to ship a scheduler that both:

- works with any standard-conforming implementation of `std::execution`, and
- performs optimally on their hardware for all of the standard algorithms.

See [Mitigating factors](#) for some reasons why this is not as terrible as it sounds.

§ 5 Mitigating factors

The loss of direct support for sender algorithm customization is a blow to power users of `std::execution`, but there are a few factors that mitigate the blow.

§ 5.1 Sender introspection

All of the senders returned from the standard algorithms are self-describing and can be unpacked into their constituent parts with structured bindings. A sufficiently motivated user can “customize” an algorithm by writing a recursive sender tree transformation, explicitly transforming senders before launching them.

§ 5.2 Third party algorithms

The sender concepts and customization points make it possible for users to write their own sender algorithms that interoperate with the standard ones. If a user wants to change the behavior of the then algorithm in some way, they have the option of writing their own and using it instead. I expect libraries of third-party algorithms to appear on GitHub in time, as they tend to.

§ 5.3 Difficulties with proprietary extensions

Some execution contexts place extra-standard requirements on the code that executes on them. For example, NVIDIA GPUs require device-accelerated code to be annotated with its proprietary `__device__` annotation. Standard libraries are unlikely to ship implementations of `std::execution` with such annotations. The consequence is that, rather than shipping just a GPU scheduler with some algorithm customizations, a vendor like NVIDIA is already committed to shipping its own complete implementation of `std::execution` (in a different namespace, of course).

For such vendors, the inability to customize standard algorithms is a moot point. Since it is *implementing* the standard algorithms, the implementations can do whatever they want.

§ 6 The removal process

§ 6.1 Approach

The approach to removing sender algorithm customization is twofold:

- Remove those components that facilitate algorithm customization and their uses where it is easy to do so.
- In all other cases, turn normative mechanisms into non-normative ones so we can change them later. This results in smaller and safer wording changes and preserves the already agreed-upon semantics in a way that is easy to verify.

§ 6.2 Procedure

The steps for removing algorithm customization are detailed below.

1. Remove the type `default_domain` (33.9.5 [\[exec.domain.default\]](#)).
2. Remove the functions:
 - `transform_sender` (33.9.6 [\[exec.snd.transform\]](#)),
 - `transform_env` (33.9.7 [\[exec.snd.transform.env\]](#)), and
 - `apply_sender` (33.9.8 [\[exec.snd.apply\]](#)).
3. Remove the query object `get_domain` (33.5.5 [\[exec.get.domain\]](#)).
4. Remove the exposition-only helpers:
 - `completion-domain` (33.9.2 [\[exec.snd.expos\]](#)/8-9),
 - `get-domain-early` (33.9.2 [\[exec.snd.expos\]](#)/13), and
 - `get-domain-late` (33.9.2 [\[exec.snd.expos\]](#)/14).

5. Change the functions `get_completion_signatures` (33.9.9 [\[exec.getcompsigs\]](#)) and `connect` (33.9.10 [\[exec.connect\]](#)) to operate on a sender determined as follows instead of passing the sender through `transform_sender`:
 - If the sender has a tag with an exposition-only *transform-sender* member function, pass the sender to this function with the receiver’s environment and continue the operation on the resulting sender. This preserves the behavior of calling `transform_sender` with the `default_domain`.
 - Otherwise, perform the operation on the passed-in sender.
6. For the following algorithms that are currently expressed in terms of a sender transformation to a lowered form, move the lowering from `alg.transform_sender(sndr, env)` to `alg.transform_sender(sndr, env)`.
 - `starts_on` (33.9.12.5 [\[exec.starts.on\]](#)),
 - `continues_on` (33.9.12.6 [\[exec.continues.on\]](#)),
 - `on` (33.9.12.8 [\[exec.on\]](#)),
 - `bulk` (33.9.12.11 [\[exec.bulk\]](#)),
 - `when_all_with_variant` (33.9.12.12 [\[exec.when.all\]](#)),
 - `stopped_as_optional` (33.9.12.14 [\[exec.stopped.opt\]](#)), and
 - `stopped_as_error` (33.9.12.15 [\[exec.stopped.err\]](#)).
7. For each sender adaptor algorithm in 33.9.12 [\[exec.adapt\]](#) that is specified to be expression-equivalent to some `transform_sender` invocation of the form:

```
transform_sender(some-computed-domain(), make-sender(tag,
  {args...}, sndr));
```

Change the expression to:

```
make-sender(tag, {args...}, sndr);
```

For example, in 33.9.12.6 [\[exec.continues.on\]](#)/3, the following:

```
transform_sender(get-domain-early(sndr), make-sender(continues_on, sch, sndr))
```

would be changed to:

```
make-sender(continues_on, sch, sndr)
```

Additionally, if there is some caveat of the form “except that `sndr` is evaluated only once,” that caveat should be removed as appropriate.

8. Merge the `schedule_from` (33.9.12.7 [\[exec.schedule.from\]](#)) and `continues_on` (33.9.12.6 [\[exec.continues.on\]](#)) algorithms into one algorithm called `continues_on`.

(Currently they are separate so that they can be customized independently; by default `continues_on` merely dispatches to `schedule_from`.)

9. Change 33.9.13.1 [[exec.sync.wait](#)] and 33.9.13.2 [[exec.sync.wait.var](#)] to dispatch directly to their default implementations instead of computing a domain and using `apply_sender` to dispatch to an implementation.
10. Fix a bug in the `on(sndr, sch, closure)` algorithm where a `write_env` is incorrectly changing the “current” scheduler before its child `continues_on` actually transfers to that scheduler. `continues_on` needs to know the scheduler on which it will be started in order to find customizations correctly in the future.
11. Tweak the wording of `parallel_scheduler` (33.15 [[exec.par.scheduler](#)]) to indicate that it (`parallel_scheduler`) is permitted to run the bulk family of algorithms in parallel in accordance with those algorithms’ semantics, rather than suggesting that those algorithms are “customized” for `parallel_scheduler`. The mechanism for such remains non-normative, however we specify the conditions under which the `parallel_scheduler` is guaranteed to run the bulk algorithms in parallel. (This is currently under-specified.)
12. Respecify `task_scheduler` in terms of `parallel_scheduler_backend` so that the bulk algorithms can be accelerated despite `task_scheduler`’s type-erasure. This addresses [LWG#4336](#). As with `parallel_scheduler`, we specify the conditions under which `task_scheduler` is guaranteed to run the bulk algorithms in parallel.
13. From the scheduler concept, remove the required expression:

```
{ auto(get_completion_scheduler<set_value_t>(get_env(schedule(std::forward<Sch>(sch))))) }  
-> same_as<remove_cvref_t<Sch>>;
```

Instead, add a semantic requirement that *if* the above expression is well-formed, then it shall compare equal to `sch`. Additionally, require that that expression is well-formed for the `parallel_scheduler`, the `task_scheduler`, and `run_loop`’s scheduler, but not `inline_scheduler`. See [inline_scheduler](#) for the motivation behind these changes, but in short: the `inline_scheduler` does not know where it completes in C++26 but will in C++29.

14. *Optional, but recommended:* Change the `env<>::query` member function to accept optional additional arguments after the query tag. This restores the original design of `env` to that which was first proposed in [[P3325R1](#)] and which was approved by LEWG straw poll in St Louis. As described in [Restoring algorithm customization in C++29](#), when asking a sender for its completion scheduler, the caller needs to pass extra information about where the operation will be started, and that will require `env<>::query` to accept extra arguments.

This is admittedly a lot of changes, but the first 9 changes represent a simplification from the *status quo*, and the other changes are either neutral in terms of specification or else

correct an existing Library issue.

In the final accounting, the result of these changes will be a vastly simpler specification for [exec].

§ 7 Restoring algorithm customization in C++29

For C++29, we want the sender algorithms in `std::execution` to be customizable, with different implementations suited for different execution contexts. If we remove customization for C++26, how do we add it back without breaking code?

Recall that many senders do not know where they will complete until they know where they will be started, and that information is not currently provided when the sender is queried for its completion scheduler. This is the shoal on which algorithm customization has foundered, because without accurate information about where operations are executing, it is impossible to pick the right algorithm implementation.

Once the problem is stated plainly, the fix (or at least a major part of it) is obvious:

When asking the sender where it will complete, *tell it where it will start*.

The implication of this is that so-called “early” customization, performed when constructing a sender, will not be coming back. The receiver’s execution environment is not known when constructing a sender. C++29 will bring back “late” customization only.

§ 7.1 Completion scheduler enhancements

A paper targetting C++29 will propose that we extend the `get_completion_scheduler` query to support an optional environment argument. Given a sender `S` and receiver `R`, the query would look like:

```
// Pass the sender's attributes and the receiver's environment
// when computing
// the completion scheduler:
auto sch = get_completion_scheduler<set_value_t>(get_env(S),
    get_env(R));
```

It will not be possible in C++26 to pass the receiver’s environment in this way, making this a conforming extension since it would not change the meaning of any existing code.

This change will also make it possible to provide a completion scheduler for the error channel in more cases. That is often not possible today since many errors are reported in-line on the context on which the operation is started. The receiver’s environment knows where the operation will be started, so by passing it to the `get_completion_scheduler<set_error_t>` query, the error completion scheduler is knowable.

Note:

The paragraph above makes it sound like this would be changing the behavior for the `get_completion_scheduler<set_error_t>(get_env(sndr))` query. But that expression will behave as it always has. Only when called with the receiver’s environment will any new behavior manifest; hence, this change is a pure extension.

By the way, this extension to `get_completion_scheduler` motivates the change to `env<>::query` described above in [The removal process](#). Although we could decide to defer that change until it is needed in C++29, it seems best to me to make the change now.

§ 7.2 Domains

There are sender expressions that complete on an indeterminate scheduler based on run-time factors; `when_all` is a good example. This is the problem the `get_domain` query solved. So long as all of `when_all`’s child senders share a common domain tag – a property of the scheduler – we know the domain on which the `when_all` operation will complete, even though we do not know which scheduler it will complete on. The [domain](#) controls algorithm selection, not the scheduler directly.

So the plan will be to bring back a `get_domain` query in C++29. Additionally, just as it is necessary to have three `get_completion_scheduler` queries, one each for the three different completion channels, it is necessary to have three `get_completion_domain` queries for the times when the completion scheduler is indeterminate but the domain is known.

Note:

Above we say, “So long as all of `when_all`’s child senders share a common domain tag [...]”. This sounds like we are adding a new requirement to the `when_all` algorithm. However, this requirement will be met for all existing uses of `when_all`. Before C++29, all senders will be in the “default” domain, so they trivially all share a common domain.

Giving a non-default domain to a scheduler is the way to opt-in to algorithm customization. Prior to C++29, there will be no `get_*domain` queries, hence the addition of those queries in C++29 will not affect any existing schedulers. And the domain queries will be so-called “forwarding” queries, meaning they will automatically be passed through layers of sender adaptors. Users will not have to change their code in order for domain information to be propagated. As a result, this change is a pure extension.

§ 7.3 Customizing connect

Since C++29 will support only late (connect-time) customization, customizing an algorithm effectively amounts to customizing that algorithm’s connect operation. By default, `connect(sndr, rcvr)` calls `sndr.connect(rcvr)`, but in C++29 there will be a way to

do something different depending on the sender’s attributes and the receiver’s environment.

connect will compute two domains, the “starting” domain and the (value) “completion” domain:

Domain kind	Query
Starting domain	<code>get_domain(get_env(rcvr))</code>
Completion domain	<code>get_completion_domain<set_value_t>(get_env(sndr), get_env(rcvr))</code>

How connect will use this information to select an algorithm implementation is currently under design. (See [Appendix A: The planned fix](#) for more information.) But at that point, it is only a matter of mechanism. The key point is that connect has the information it needs to dispatch accurately, and that we can make that addition without breaking code. And we can.

§ 7.4 The parallel and task schedulers and bulk

Once we have a general mechanism for customizing algorithms, we can consider changing `parallel_scheduler` and `task_scheduler` to use that mechanism to find parallel implementations of the bulk algorithms. In C++26, it is unspecified precisely how those schedulers accelerate bulk, and we can certainly leave it that way for C++29. No change is often the safest change and always the easiest.

If we wanted to switch to using the new algorithm dispatch mechanics in C++29, I believe we can do so with minimal impact on existing code. Any behavior change would be an improvement, accelerating bulk operations that *should* have been accelerated but were not.

Consider the following sender:

```
starts_on(parallel_scheduler(), just() | bulk(fn))
```

In C++26, we can offer no iron-clad standard guarantee that this bulk operation will be accelerated even though it is executing on the parallel scheduler. The predecessor of `bulk`, `just()`, does not know where it will complete in C++26. There is no plumbing yet to tell it that it will be started on the parallel scheduler. As a result, it is QoI whether this bulk will execute in parallel or not.

But suppose we add a `get_completion_domain<set_value_t>` query to the `parallel_scheduler` such that the query returns an instance of a new type: `parallel_domain`. Now, when connecting the bulk sender, connect will ask for the predecessor’s domain, passing also the receiver’s environment. Now the `just()` sender is able to say where it completes: the domain where it starts, `get_domain(get_env(rcvr))`. This will return `parallel_domain{}`. connect would then use that information to find a parallel implementation of `bulk`.

As a result, in C++29 we could guarantee that this usage of `bulk` will be parallelized. For some `stdlib` implementations, this would be a behavior change: what once executed serially on a thread of the parallel scheduler now executes in parallel on many threads. Can that break working code? Yes, but only code that had already violated the preconditions of `bulk`: that `fn` can safely be called in parallel.

I do not believe this should be considered a breaking change, since any code that breaks is already broken.

All of the above is true also for `task_scheduler`, which merely adds an indirection to the call to `connect`. After the changes suggested by this paper, the `task_scheduler` accelerates `bulk` in the same way as `parallel_scheduler`.

Note:

If we assign `parallel_domain` to the `parallel_scheduler`, and we *also* add a requirement to `when_all` that all of its child operations share a common domain (see [Domains](#)), does that have the potential to break existing code? It would not. We would make `parallel_domain` inherit from `default_domain` so that `when_all` will compute the common domain as `default_domain` even if one child completes in the `parallel_domain`.

§ 7.5 `inline_scheduler`

The suggestion above to extend the `get_completion_scheduler<*>` query presents an intriguing possibility for the `inline_scheduler`: the ability for it to report the scheduler on which its scheduling operations complete!

Consider the sender `schedule(inline_scheduler{})`. Ask it where it completes today and it will say, “I complete on the `inline_scheduler`.”, which isn’t terribly useful. However, if you ask it, “Where will you complete – and by the way you will be started on the `parallel_scheduler`?”, now that sender can report that it will complete on the `parallel_scheduler`.

The result is that code that uses the `inline_scheduler` will no longer cause the *actual* scheduler to be hidden.

This realization is the motivation behind the change to strike the `get_completion_scheduler<set_value_t>(get_env(schedule(sch)))` requirement from the scheduler concept. We want that expression to be ill-formed for the `inline_scheduler`. Instead, we want the following query to be well-formed (in C++29):

```
get_completion_scheduler<set_value_t>(get_env(schedule(inline_scheduler{})), get_env(rcvr))
```

That expression should be equivalent to `get_scheduler(get_env(rcvr))`, which says that the sender of `inline_scheduler` completes wherever it is started.

Note:

The reason we do not want `inline_scheduler` to have a (largely meaningless) completion scheduler in C++26 is because we want it to have a meaningful one in C++29. And it would be strange if asking for the completion scheduler gave different answers depending on whether or not an environment was passed to the query.

This follows the general principle that if you query a sender's metadata early (sans environment) and then later query it again with an environment, the answer should not change. If the sender does not know the answer with certainty without an environment, better for the expression to be ill-formed rather than returning potentially inaccurate information.

§ 8 Proposed wording

[Editor's note: In 33.4 [execution.syn], make the following changes:]

... as before ...

```
namespace std::execution {
    // [exec.queries], queries
struct get_domain_t { unspecified };
    struct get_scheduler_t { unspecified };
    struct get_delegation_scheduler_t { unspecified };
    struct get_forward_progress_guarantee_t { unspecified };
    template<class CP0>
        struct get_completion_scheduler_t { unspecified };
    struct get_await_completion_adaptor_t { unspecified };

inline constexpr get_domain_t get_domain{};
    inline constexpr get_scheduler_t get_scheduler{};
        inline constexpr get_delegation_scheduler_t
            get_delegation_scheduler{};
    enum class forward_progress_guarantee;
        inline constexpr get_forward_progress_guarantee_t
            get_forward_progress_guarantee{};
    template<class CP0>
        constexpr get_completion_scheduler_t<CP0>
            get_completion_scheduler{};
        inline constexpr get_await_completion_adaptor_t
            get_await_completion_adaptor{};
```

... as before ...

```
// [exec.env], class template env
template<queryable... Envs>
    struct env;

// [exec.domain.default], execution domains
struct default_domain;
```



```

// [exec.sched], schedulers
struct scheduler_t {};

... as before ...

template<sender Sndr>
using tag_of_t = see below;

// [exec.snd.transform], sender transformations
template<class Domain, sender Sndr, queryable... Env>
requires (sizeof...(Env) <= 1)
constexpr sender decltype(auto) transform_sender(
Domain dom, Sndr&& sndr, const Env&... env) noexcept(see
below);

// [exec.snd.transform.env], environment transformations
template<class Domain, sender Sndr, queryable Env>
constexpr queryable decltype(auto) transform_env(
Domain dom, Sndr&& sndr, Env&& env) noexcept;

// [exec.snd.apply], sender algorithm application
template<class Domain, class Tag, sender Sndr, class... Args>
constexpr decltype(auto) apply_sender(
Domain dom, Tag, Sndr&& sndr, Args&&... args) noexcept(see
below);

// [exec.connect], the connect sender algorithm
struct connect_t;
inline constexpr connect_t connect{};

... as before ...

```

[Editor's note: Remove subsection 33.5.5 [exec.get.domain].]

[Editor's note: In 33.6 [exec.sched], change paragraphs 1 and 5 and strike paragraph 6 as follows:]

1. The scheduler concept defines the requirements of a scheduler type (33.3 [exec.async.ops]). `schedule` is a customization point object that accepts a scheduler. A valid invocation of `schedule` is a `schedule-expression`.

```

namespace std::execution {
    template<class Sch>
    concept scheduler =
        derived_from<typename remove_cvref_t<Sch>::scheduler_concept, scheduler_t> &&
        queryable<Sch> &&
        requires(Sch&& sch) {
            { schedule(std::forward<Sch>(sch)) } ->
            sender;
    { auto(get_completion_scheduler<set_value_t>)

```

```

        get_env(schedule(std::forward<Sch>
        (sch)))))) ->
        same_as<remove_cvref_t<Sch>>;
    } &&
    equality_comparable<remove_cvref_t<Sch>>
    &&
    copyable<remove_cvref_t<Sch>>;
}

```

... as before ...

5. For a given scheduler expression `sch`, if the expression `auto(get_completion_scheduler<set_value_t>(get_env(schedule(sch)))` is well-formed, it shall have type `remove_cvref_t<Sch>` and shall compare equal to `sch`.
- ~~6. For a given scheduler expression `sch`, if the expression `get_domain(sch)` is well-formed, then the expression `get_domain(get_env(schedule(sch)))` is also well-formed and has the same type.~~

[Editor's note: In 33.9.1 [exec.snd.general], change paragraph 1 as follows:]

1. Subclauses 33.9.11 [exec.factories] and 33.9.12 [exec.adapt] define ~~customizable~~ algorithms that return senders. ~~Each algorithm has a default implementation.~~ Let `sndr` be the result of an invocation of such an algorithm or an object equal to the result (18.2 [concepts.equality]), and let `Sndr` be `decltype((sndr))`. Let `rcvr` be a receiver of type `Rcvr` with associated environment `env` of type `Env` such that `sender_to<Sndr, Rcvr>` is `true`. ~~For the default implementation of the algorithm that produced `sndr`, e~~Connecting `sndr` to `rcvr` and starting the resulting operation state (33.3 [exec.async.ops]) necessarily results in the potential evaluation (6.3 [basic.def.odr]) of a set of completion operations whose first argument is a subexpression equal to `rcvr`. Let `Sigs` be a pack of completion signatures corresponding to this set of completion operations, and let `CS` be the type of the expression `get_completion_signatures<Sndr, Env>()`. Then `CS` is a specialization of the class template `completion_signatures` (33.10 [exec.cmplsig]), the set of whose template arguments is `Sigs`. If none of the types in `Sigs` are dependent on the type `Env`, then the expression `get_completion_signatures<Sndr>()` is well-formed and its type is `CS`. ~~If a user-provided implementation of the algorithm that produced `sndr` is selected instead of the default:~~

- (1.1)
- ~~Any completion signature that is in the set of types denoted by `completion_signatures_of_t<Sndr, Env>` and that is not part of `Sigs` shall correspond to error or stopped completion operations, unless otherwise specified.~~

- (1.2) — ~~If none of the types in Sigs are dependent on the type Env, then completion_signatures_of_t<Sndr> and completion_signatures_of_t<Sndr, Env> shall denote the same type.~~

[Editor's note: Change 33.9.2 [exec.snd.expos] paragraph 6 as follows:]

6. For a scheduler `sch`, `SCHED-ATTRS(sch)` is ~~an expression o1 whose type satisfies~~ queryable ~~such that o1.query(get_completion_scheduler<Tag>) is an expression with the same type and value as sch~~ equivalent to MAKE-ENV(get_completion_scheduler<set value t>, sch) ~~where Tag is one of set_value_t or set_stopped_t, and such that o1.query(get_domain) is expression-equivalent to sch.query(get_domain).~~ `SCHED-ENV(sch)` is ~~an expression o2 whose type satisfies~~ queryable ~~such that o2.query(get_scheduler) is a prvalue with the same type and value as sch, and such that o2.query(get_domain) is expression-equivalent to sch.query(get_domain)~~ equivalent to MAKE-ENV(get_scheduler, sch).

[Editor's note: Remove the prototype of the exposition-only `completion-domain` function just before 33.9.2 [exec.snd.expos] paragraph 8, and with it remove paragraphs 8 and 9, which specify the function's behavior.]

[Editor's note: Remove 33.9.2 [exec.snd.expos] paragraphs 13 and 14 and the prototypes for the `get-domain-early` and `get-domain-late` functions.]

[Editor's note: Remove subsection 33.9.5 [exec.domain.default].]

[Editor's note: Remove subsection 33.9.6 [exec.snd.transform].]

[Editor's note: Remove subsection 33.9.7 [exec.snd.transform.env].]

[Editor's note: Remove subsection 33.9.8 [exec.snd.apply].]

[Editor's note: Change 33.9.9 [exec.getcomplsigs] as follows:]

1. Let `except` be an rvalue subexpression of an unspecified class type `Except` such that `move_constructible<Except> && derived_from<Except, exception> is true`. Let `CHECKED-COMPLSIGS(e)` be `e` if `e` is a core constant expression whose type satisfies `valid-completion-signatures`; otherwise, it is the following expression:

```
(e, throw except, completion_signatures())
```

Let `get-complsigs<Sndr, Env...>()` be expression-equivalent to `remove_reference_t<Sndr>::template get_completion_signatures<Sndr, Env...>()`. ~~Let NewSndr be Sndr if sizeof...(Env) ==~~

~~0 is true; otherwise, decltype(s) where s is the following expression:~~
 Let `NewSndr` be `decltype(tag of t<Sndr>().transform-`
`sender(declval<Sndr>(), declval<Env>()....))` if that expression
 is well-formed, and `Sndr` otherwise.

```
transform_sender{
  get_domain_late(declval<Sndr>(), declval<Env>
    (...)),
  declval<Sndr>(),
  declval<Env>()....}
```

2. Constraints: `sizeof...(Env) <= 1` is true.

3. Effects: Equivalent to: *... as before ...*

[Editor's note: Change 33.9.10 [exec.connect] as follows:]

1. connect connects (33.3 [exec.async.ops]) a sender with a receiver.
2. The name connect denotes a customization point object. For subexpressions `sndr` and `rcvr`, let `Sndr` be `decltype((sndr))` and `Rcvr` be `decltype((rcvr))`; let `new_sndr` be the expression ~~`transform_sender(decltype(get_domain_late(sndr, get_env(rcvr))) {},`~~
~~`sndr, get_env(rcvr))`~~ `tag of t<Sndr>().transform-`
`sender(sndr, get_env(rcvr))` if that expression is well-formed, and
`sndr` otherwise; and let `DS` and `DR` be `decay_t<decltype((new_s-`
`ndr))>` and `decay_t<Rcvr>`, respectively.
3. Let `connect-awaiter-promise` be *... as before ...*

[Editor's note: Change 33.9.11.1 [exec.schedule] paragraph 4 as follows:]

4. If the expression

```
get_completion_scheduler<set_value_t>(get_env(sch.sched-  

ule())) == sch
```

is ~~ill-formed or~~ well-formed and does not evaluates to ~~false~~ sch, the behavior of calling `schedule(sch)` is undefined.

[Editor's note: From 33.9.12.1 [exec.adapt.general], strike paragraph (3.6) as follows:]

3. Unless otherwise specified:

... as before ...

- (3.5) — An adaptor whose child senders are all non-dependent (33.3 [exec.async.ops]) is itself non-dependent.
- (3.6) — ~~These requirements apply to any function that is selected by the implementation of the sender adaptor.~~

- (3.7) — *Recommended practice:* Implementations should use the completion signatures of the adaptors to communicate type errors to users and to propagate any such type errors from child senders.

[Editor's note: Change 33.9.12.5 [exec.starts.on] paragraph 3 as follows:]

3. Otherwise, the expression `starts_on(sch, sndr)` is expression-equivalent to: `make_sender(starts_on, sch, sndr)`.

```
transform_sender(  
    query_with_default(get_domain, sch,  
        default_domain()),  
    make_sender(starts_on, sch, sndr))
```

~~except that `sch` is evaluated only once.~~

4. Let `out_sndr` and `env` be subexpressions such that `OutSndr` is `decltype(out_sndr)`. If `sender_for<OutSndr, starts_on_t>` is `false`, then the ~~expressions `starts_on.transform_env(out_sndr, env)` and `expression starts_on.transform_sender`~~ `starts_on.transform_sender` ~~`transform_sender(out_sndr, env)` are~~ is ill-formed; otherwise it is equivalent to:

- (4.1) — ~~`starts_on.transform_env(out_sndr, env)` is equivalent to:~~

```
auto&& [_, sch, _] = out_sndr;  
return JOIN_ENV(SCHED_ENV(sch), FWD-  
    ENV(env));
```

- (4.2) — ~~`starts_on.transform_sender(out_sndr, env)` is equivalent to:~~

```
auto&& [_, sch, sndr] = out_sndr;  
return let_value(  
    schedule(sch),  
    [sndr = std::forward_like<OutSndr>(sndr)]  
    () mutable  
        noexcept(is_nothrow_move_con-  
            structible_v<decay_t<OutSndr>>) {  
        return std::move(sndr);  
    });
```

5. Let `out_sndr` be *...as before ...*

[Editor's note: Remove subsection 33.9.12.6 [exec.continues.on]]

[Editor's note: Change 33.9.12.7 [exec.schedule.from] to [exec.continues.on] and change it as follows:]

33.9.12.76 execution::`schedule_fromcontinues_on` [exec:~~`sched- ule.fromcontinues.on`~~]

1. ~~`schedule_fromcontinues_on`~~ schedules work dependent on the completion of a sender onto a scheduler's associated execution resource.

~~{Note 1: `schedule_from` is not meant to be used in user code; it is used in the implementation of `continues_on`. — end note}~~

2. The name `schedule_fromcontinues_on` denotes a customization point object. For some subexpressions `sch` and `sndr`, let `Sch` be `decltype((sch))` and `Sndr` be `decltype((sndr))`. If `Sch` does not satisfy scheduler, or `Sndr` does not satisfy sender, `schedule_from(sch, sndr)continues_on(sndr, sch)` is ill-formed.
3. Otherwise, the expression `schedule_from(sch, sndr)continues_on(sndr, sch)` is expression-equivalent to: `make_sender(continues_on, sch, sndr)`

```
transform_sender(  
    query_with_default(get_domain, sch,  
        default_domain()),  
    make_sender(schedule_from, sch, sndr))
```

~~except that `sch` is evaluated only once.~~

4. The exposition-only class template `impls_for` (33.9.1 [exec.snd.gener-al]) is specialized for `schedule_from_tcontinues_on_t` as follows:

```
namespace std::execution {  
    template<>  
        struct impls_for<schedule_from_tcontin-  
            ues_on_t> : default_impls {  
            static constexpr auto get_attrs = see  
                below;  
            static constexpr auto get_state = see  
                below;  
            static constexpr auto complete = see  
                below;  
  
            template<class Sndr, class... Env>  
                static consteval void check_types();  
        };  
}
```

5. The member `impls_for<schedule_from_tcontinues_on_t>::get_attrs` is initialized with a callable object equivalent to the following lambda:

```
[(const auto& data, const auto& child) noexcept  
    -> decltype(auto) {  
    return JOIN_ENV(SCHED_ATTRS(data), FWD-  
        ENV(get_env(child)));  
}]
```

6. The member `impls_for<schedule_from_tcontinues_on_t>::get_state` is initialized with a callable object equivalent to the following lambda:

...as before ...

```
template<class Sndr, class... Env>
static consteval void check-types();
```

...as before ...

12. The member `impls-for<schedule_from_tcontinues_on_t>::complete` is initialized with a callable object equivalent to the following lambda:

...as before ...

13. Let `out_sndr` be a subexpression denoting a sender returned from `schedule_from(sch, sndr)continues_on(sndr, sch)` or one equal to such, and let `OutSndr` be the type `decltype((out_sndr))`. Let `out_rcvr` be ...as before ...

[Editor's note: Change 33.9.12.8 [exec.on] paragraphs 3-8 as follows:]

3. Otherwise, if `decltype((sndr))` satisfies sender, the expression `on(sch, sndr)` is expression-equivalent to: `make_sender(on, sch, sndr)`.

```
transform_sender({
    query_with_default(get_domain, sch,
        default_domain()),
    make_sender(on, sch, sndr)})
```

~~except that sch is evaluated only once.~~

4. For subexpressions `sndr`, `sch`, and closure, if

(4.1) — `decltype((sch))` does not satisfy scheduler, or

(4.2) — `decltype((sndr))` does not satisfy sender, or

(4.3) — closure is not a pipeable sender adaptor closure object ([exec.adapt.obj]), the expression `on(sndr, sch, closure)` is ill-formed; otherwise, it is expression-equivalent to: `make_sender(on, product-type{sch, closure}, sndr)`.

```
transform_sender({
    get_domain_early(sndr),
    make_sender(on, product-type{sch,
        closure}, sndr)})
```

~~except that sndr is evaluated only once.~~

5. Let `out_sndr` and `env` be subexpressions, let `OutSndr` be `decltype((out_sndr))`, and let `Env` be `decltype((env))`. If `sender-for<OutSndr, on_t>` is `false`, then the ~~expressions-on-transform~~

~~m_env(out_sndr, env)~~ and ~~expression on. transform_sender~~transform_sender(out_sndr, env) ~~are~~ is ill-formed.

6. Otherwise: Let `not-a-scheduler` be an unspecified empty class type.
7. The expression ~~on. transform_env(out_sndr, env)~~ has effects equivalent to:

```
auto&& [_, data, _] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
    return JOIN_ENV(SCHED-
        ENV(std::forward_like<OutSndr>(data)),
        FWD_ENV(std::forward<Env>(env)));
} else {
    return std::forward<Env>(env);
}
```

8. The expression ~~on. transform_sender~~transform_sender(out_sndr, env) has effects equivalent to:

```
auto&& [_, data, child] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
    auto orig_sch =
        query-with-default(get_scheduler, env,
            not-a-scheduler());

    if constexpr (same_as<decltype(orig_sch),
        not-a-scheduler>) {
        return not-a-sender{};
    } else {
        return continues_on(
            starts_on(std::forward_like<OutSndr>
                (data), std::forward_like<OutSndr>
                (child)),
            std::move(orig_sch));
    }
} else {
    auto& [sch, closure] = data;
    auto orig_sch = query-with-default(
        get_completion_scheduler<set_value_t>,
        get_env(child),
        query-with-default(get_scheduler, env,
            not-a-scheduler()));

    if constexpr (same_as<decltype(orig_sch),
        not-a-scheduler>) {
        return not-a-sender{};
    } else {
        return write_env continues_on(
            continues_on write_env(
                std::forward_like<OutSndr>(closure)(
                    continues_on(
                        write_env(std::forward_-
                            like<OutSndr>(child), SCHED-
                            ENV(orig_sch)),
```



```

sch)),
orig_sch SCHED-ENV(sch)),
SCHED-ENV(sch) orig_sch);
}
}

```

[Editor's note: Change 33.9.12.9 [exec.then] paragraph 3 as follows:]

3. Otherwise, the expression *then-cpo*(sndr, f) is expression-equivalent to: *make-sender(then-cpo, f, sndr)*.

```

transform_sender(get_domain_early(sndr), make-
sender(then-cpo, f, sndr))

```

~~except that *sndr* is evaluated only once.~~

[Editor's note: Change 33.9.12.10 [exec.let] paragraphs 2-4 as follows:]

2. For *let_value*, *let_error*, and *let_stopped*, let *set-cpo* be *set_value*, *set_error*, and *set_stopped*, respectively. Let the expression *let-cpo* be one of *let_value*, *let_error*, or *let_stopped*. For a sub-expression *sndr*, let *let-env*(*sndr*) be expression-equivalent to the first well-formed expression below:

(2.1) — *SCHED-ENV*(*get_completion_scheduler*<*decayed-typeof*<*set-cpo*>>(get-env(*sndr*)))

~~(2.2) — MAKE-ENV(get-domain, get-domain(get-env(sndr)))~~

(2.3) — (*void*(*sndr*), env<>{})

3. The names *let_value*, *let_error*, and *let_stopped* denote *... as before ...*
4. Otherwise, the expression *let-cpo*(*sndr*, f) is expression-equivalent to: *make-sender(let-cpo, f, sndr)*.

```

transform_sender(get_domain_early(sndr), make-
sender(let-cpo, f, sndr))

```

~~except that *sndr* is evaluated only once.~~

[Editor's note: Change 33.9.12.11 [exec.bulk] paragraphs 3 and 4 and insert paragraphs 5 and 6 as follows:]

3. Otherwise, the expression *bulk-algo*(*sndr*, *policy*, *shape*, f) is expression-equivalent to:

```

transform_sender(get_domain_early(sndr), make-sender(
bulk-algo, product-type<see below, Shape, Func>{policy,
shape, f}, sndr))

```

~~except that sndr is evaluated only once.~~ The first template argument of *product-type* is *Policy* if *Policy* models *copy_constructible*, and *const Policy* otherwise.

4. Let *sndr* ~~and env be subexpressions~~ be an expression such that *Sndr* is *decltype((sndr))*. If *sender-for*<*Sndr*, *bulk_t*> is *false*, then the expression ~~*bulk.transform_sender(sndr, env)*~~ *as-bulk-chunked(sndr)* is ill-formed; otherwise, it is equivalent to:

```
auto [_, data, child] = sndr;
auto& [policy, shape, f] = data;
auto new_f = [func = std::move(f)](Shape begin,
    Shape end, auto&&... vs)
    noexcept(noexcept(f(begin, vs...))) {
    while (begin != end)
        func(begin++, vs...);
}
return bulk_chunked(std::move(child), policy,
    shape, std::move(new_f));
```

~~[Note: This causes the *bulk(sndr, policy, shape, f)* sender to be expressed in terms of *bulk_chunked(sndr, policy, shape, f)* when it is connected to a receiver whose execution domain does not customize *bulk*.—end note]~~

5. Let *sndr* and *env* be subexpressions, let *Sndr* be *decltype((sndr))*, and let *sch* be expression-equivalent to *get_completion_scheduler<set_value_t>(get_env(sndr.get<2>()))*. If *sender-for*<*Sndr*, *decayed-typeof*<*bulk_algo*>> is *false*, the expression *bulk_algo.transform_sender(sndr, env)* is ill-formed; otherwise, it is expression-equivalent to:

- (6.1) — *sch.bulk-transform(sndr, env)* if that expression is well-formed; otherwise,
- (6.2) — *sch.bulk-transform(as-bulk-chunked(sndr), env)* if that expression is well-formed; otherwise,
- (6.3) — *bulk_algo.transform_sender(sndr, env)* is ill-formed.

[Editor's note: Change 33.9.12.12 [exec.when.all] as follows:]

1. *when_all* and *when_all_with_variant* both ...*as before* ...
2. The names *when_all* and *when_all_with_variant* denote customization point objects. Let *sndrs* be a pack of subexpressions; and let *Sndrs* be a pack of the types *decltype((sndrs))...*; ~~and let *CD* be the type *common_type_t*<*decltype(get_domain_early(sndrs))...*>. Let *CD2* be *CD* if *CD* is well-formed, and *default_domain* otherwise.~~ The expres-

sions `when_all(sndrs...)` and `when_all_with_variant(sndrs...)` are ill-formed if any of the following is `true`:

- (2.1) — `sizeof...(sndrs)` is `0`, or
- (2.2) — `(sender<Sndrs> && ...)` is `false`.

3. The expression `when_all(sndrs...)` is expression-equivalent to:
`make_sender(when_all, {}, sndrs...)`.

```
transform_sender(CD2(), make_sender(when_all,  
{}), sndrs...))
```

4. The exposition-only class template `impls_for` (33.9.1 [exec.snd.gener-al]) is specialized for `when_all_t` as follows:

```
namespace std::execution {  
    template<>  
        struct impls_for<when_all_t> : default_impls  
        {  
            static constexpr auto get_attrs = see  
            below;  
            static constexpr auto get_env = see below;  
            static constexpr auto get_state = see  
                below;  
            static constexpr auto start = see below;  
            static constexpr auto complete = see  
                below;  
  
            template<class Sndr, class... Env>  
                static constexpr void check_types();  
        };  
}
```

...as before ...

9. *Throws*: Any exception thrown as a result of evaluating the *Effects*, ~~or~~
~~an exception of an unspecified type derived from exception when CD is~~
~~ill-formed.~~

10. The member ~~`impls_for<when_all_t>::get_attrs`~~ is initialized with a
callable object equivalent to the following lambda expression:

```
{}(auto&&, auto&&... child) noexcept {  
    if constexpr (same_as<CD, default_domain>) {  
        return env<>();  
    } else {  
        return MAKE_ENV(get_domain, CD());  
    }  
}
```

...as before ...

19. The expression `when_all_with_variant(sndrs...)` is expression-equivalent to: `make_sender(when_all_with_variant, {}, sndrs...)`.

```
transform_sender(CD2(), make_sender(when_all_with_variant, {}, sndrs...));
```

20. Given subexpressions `sndr` and `env`, if `sender_for<decltype((sndr))`, `when_all_with_variant_t` is `false`, then the expression `when_all_with_variant.transform_sender.transform_sender(sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto&& [_, _, ...child] = sndr;
return when_all(into_variant(std::forward_like<decltype((sndr))>(child))...);
```

[Note 1: This causes the `when_all_with_variant(sndrs...)` sender to become `when_all(into_variant(sndrs)...) when it is connected with a receiver whose execution domain does not customize when_all_with_variant`. — end note]

[Editor's note: Change 33.9.12.13 [exec.into.variant] paragraph 3 as follows:]

3. Otherwise, the expression `into_variant(sndr)` is expression-equivalent to: `make_sender(into_variant, {}, sndr)`.

```
transform_sender(get_domain_early(sndr), make_sender(into_variant, {}, sndr))
```

~~except that `sndr` is only evaluated once.~~

[Editor's note: Change 33.9.12.14 [exec.stopped.opt] paragraphs 2 and 4 as follows:]

2. The name `stopped_as_optional` denotes a pipeable sender adaptor object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. The expression `stopped_as_optional(sndr)` is expression-equivalent to: `make_sender(stopped_as_optional, {}, sndr)`.

```
transform_sender(get_domain_early(sndr), make_sender(stopped_as_optional, {}, sndr))
```

~~except that `sndr` is only evaluated once.~~

3. The exposition-only class template *impls-for ... as before ...*
4. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))` and `Env` is `decltype((env))`. If `sender_for<Sndr, stopped_as_optional_t>` is `false` then the expression `stopped_as_optional.transform_sender.transform_sender(sndr, env)` is ill-formed; otherwise, if `sender_in<child-type<Sndr>, FWD-ENV-T(Env)>` is `false`, the expression `stopped_as_optional.trans-`

~~form_sender~~transform_sender(sndr, env) is equivalent to *not-a-sender*(); otherwise, it is equivalent to:

```
auto&& [_, _, child] = sndr;
using V = single_sender_value_type<child-
    type<Sndr>, FWD-ENV-T(Env)>;
return let_stopped(
    then(std::forward_like<Sndr>(child),
        [<class... Ts>(Ts&&... ts) noexcept
            (is_nothrow_constructible_v<V,
                Ts...>) {
                return optional<V>(in_place,
                    std::forward<Ts>(ts)...);
            }
        ),
    []() noexcept { return just(optional<V>());
    });
```

[Editor's note: Change 33.9.12.15 [exec.stopped.err] paragraphs 2 and 3 as follows:]

2. The name `stopped_as_error` denotes a pipeable sender adaptor object. For some subexpressions `sndr` and `err`, let `Sndr` be `decltype((sndr))` and let `Err` be `decltype((err))`. If the type `Sndr` does not satisfy `sender` or if the type `Err` does not satisfy `movable-value`, `stopped_as_error(sndr, err)` is ill-formed. Otherwise, the expression `stopped_as_error(sndr)` is expression-equivalent to: `make_sender(stopped_as_error, err, sndr)`.

~~`transform_sender(get_domain_early(sndr), make_sender(stopped_as_error, err, sndr))`~~

~~except that `sndr` is only evaluated once.~~

3. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))` and `Env` is `decltype((env))`. If `sender_for<Sndr, stopped_as_error_t>` is `false` then the expression `stopped_as_error`.~~`transform_sender`~~`transform_sender`(`sndr, env`) is ill-formed; otherwise, it is equivalent to:

```
auto&& [_, err, child] = sndr;
using E = decltype(auto(err));
return let_stopped(
    std::forward_like<Sndr>(child),
    [err = std::forward_like<Sndr>(err)]() noexcept
        (is_nothrow_move_constructible_v<E>) {
        return just_error(std::move(err));
    });
```

[Editor's note: Change 33.9.12.16 [exec.associate] paragraph 10 as follows:]

10. The name `associate` denotes a pipeable sender adaptor object. For subexpressions `sndr` and `token`:

- (10.1) — If `decltype((sndr))` does not satisfy sender, or `remove_cvref_t<decltype((token))>` does not satisfy scope_token, then `associate(sndr, token)` is ill-formed.
- (10.2) — Otherwise, the expression `associate(sndr, token)` is expression-equivalent to: `make_sender(associate, associate_data(token, sndr))`.

```
transform_sender(get_domain_early(sndr),
                 make_sender(associate, as-
                           sociate_data(token, sndr)))
```

~~except that `sndr` is evaluated only once.~~

[Editor's note: Change 33.9.13.1 [exec.sync.wait] paragraphs 4 and 9 as follows:]

4. The name `this_thread::sync_wait` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. The expression `this_thread::sync_wait(sndr)` is expression-equivalent to the following, except that `sndr` is evaluated only once: `sync_wait.apply(sndr)`, where `apply` is the exposition-only member function specified below.

```
apply_sender(get_domain_early(sndr), sync_wait,
             sndr)
```

Mandates:

- (4.1) — `sender_in<Sndr, sync_wait_env>` is true.
- (4.2) — The type `sync_wait_result_type<Sndr>` is well-formed.
- ~~(4.3) — `same_as<decltype(e), sync_wait_result_type<Sndr>>` is true, where `e` is the `apply_sender` expression i>~~

... as before ...

9. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. If `sender_to<Sndr, sync_wait_receiver<Sndr>>` is `false`, the expression `sync_wait.apply_senderapply(sndr)` is ill-formed; otherwise, it is equivalent to:

```
sync_wait_state<Sndr> state;
auto op = connect(sndr, sync_wait_receiver<Sndr>{&state});
start(op);

state.loop.run();
if (state.error) {
    rethrow_exception(std::move(state.error));
}
return std::move(state.result);
```

[Editor's note: Change *Note 1* in 33.9.13.1 [exec.sync.wait] paragraph 10.1 as follows:]

[*Note 1*: The **default** implementation of `sync_wait` achieves forward progress guarantee delegation by providing a `run_loop` scheduler via the `get_delegation_scheduler` query on the *sync-wait-receiver's* environment. The `run_loop` is driven by the current thread of execution. — *end note*]

[Editor's note: Change 33.9.13.2 [exec.sync.wait.var] paragraphs 1 and 2 as follows:]

1. The name `this_thread::sync_wait_with_variant` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype(into_variant(sndr))`. The expression `this_thread::sync_wait_with_variant(sndr)` is expression-equivalent to ~~the following, except `sndr` is evaluated only once:~~ `sync_wait_with_variant.apply(sndr)`, where `apply` is the exposition-only member function specified below.

~~`apply_sender(get_domain_early(sndr), sync_wait_with_variant, sndr)`~~

Mandates:

- (1.1) — `sender_in<Sndr, sync-wait-env>` is `true`.
 - (1.2) — The type `sync-wait-with-variant-result-type<Sndr>` is well-formed.
 - ~~(1.3) — `same_as<decltype(e), sync-wait-with-variant-result-type<Sndr>>` is `true`, where `e` is the `apply_sender` expression i~~
2. The expression `sync_wait_with_variant.apply_sender``apply`(`sndr`) is equivalent to:

```
using result_type = sync-wait-with-variant-re-
                    sult-type<Sndr>;
if (auto opt_value = sync_wait(into_vari-
    ant(sndr))) {
    return result_type(std::move(get<0>(*opt_val-
        ue)));
}
return result_type(nullopt);
```

[Editor's note: Change *Note 1* in 33.9.13.1 [exec.sync.wait] paragraph 10.1 as follows:]

[*Note 1*: The **default** implementation of `sync_wait_with_variant` achieves forward progress guarantee delegation (6.10.2.3 [intro.progress]) by relying on the forward progress guarantee delegation provided by `sync_wait`. — *end note*]

[Editor's note: Change 33.11.2 [exec.env] as follows:]

```

namespace std::execution {
    template<queryable... Envs>
    struct env {
        Envs0 envs0;           // exposition only
        Envs1 envs1;           // exposition only
        :
        Envsn-1 envsn-1;       // exposition only

        template<class QueryTag, class... Args>
        constexpr decltype(auto) query(QueryTag q,
            Args&&... args) const noexcept(see below);
    };

    template<class... Envs>
    env(Envs...) -> env<unwrap_reference_t<Envs>...>;
}

```

1. The class template `env` is used to construct a queryable object from several queryable objects. Query invocations on the resulting object are resolved by attempting to query each subobject in lexical order.

...as before ...

```

template<class QueryTag, class... Args>
constexpr decltype(auto) query(QueryTag q, Args&&...
    args) const noexcept(see below);

```

5. Let *has-query* be the following exposition-only concept:

```

template<class Env, class QueryTag, class...
    Args>
concept has-query =           // expo-
    sition only
    requires (const Env& env, Args&&... args) {
        env.query(QueryTag(), std::forward<Args>
            (args)...);
    };

```

6. Let *fe* be the first element of `envs0, envs1, ... envsn-1` such that the expression `fe.query(q, std::forward<Args>(args)...)` is well-formed.
7. *Constraints*: `(has-query<Envs, QueryTag, Args...> || ...)` is `true`.
8. *Effects*: Equivalent to: `return fe.query(q, std::forward<Args>(args)...) ;`
9. *Remarks*: The expression in the `noexcept` clause is equivalent to `noexcept(fe.query(q, std::forward<Args>(args)...) )`.

[Editor's note: In 33.12.1.2 [exec.run.loop.types], add a new paragraph after paragraph 4 as follows:]

4. Let *sch* be an expression of type *run-loop-scheduler*. The expression `schedule(sch)` has type *run-loop-sender* and is not potentially-throwing if *sch* is not potentially-throwing.
5. For type *set-tag* other than `set_error_t`, the expression `get_completion_scheduler<set-tag>(get_env(schedule(sch))) == sch` evaluates to `true`.

[Editor's note: Change 33.13.3 [exec.affine.on] paragraph 3 as follows:]

3. Otherwise, the expression `affine_on(sndr, sch)` is expression-equivalent to: `make_sender(affine_on, sch, sndr)`.

```
transform_sender(get_domain_early(sndr), make_sender(affine_on, sch, sndr))
```

~~except that `sndr` is evaluated only once.~~

[Editor's note: Change paragraph 3 of 33.13.4 [exec.inline.scheduler] as follows:]

3. Let *sndr* be an expression of type *inline-sender*, let *rcvr* be an expression such that `receiver_of<decltype((rcvr))>, CS>` is `true` where *CS* is `completion_signatures<set_value_t()>`, then: [Editor's note: Move the text of (3.1) below into this paragraph.]

(3.1) the expression `connect(sndr, rcvr)` has type *inline-state*<`remove_cvref_t<decltype((rcvr))>>` and is potentially-throwing if and only if `((void)sndr, auto(rcvr))` is potentially-throwing, ~~and~~.

(3.2) ~~the expression `get_completion_scheduler<set_value_t>(get_env(sndr))` has type *inline_scheduler* and is potentially-throwing if and only if `get_env(sndr)` is potentially-throwing.~~

[Editor's note: Change 33.13.5 [exec.task.scheduler] as follows:]

```
namespace std::execution {
    class task_scheduler {
        class ts_sender; // exposition only

        template<receiver R>
            class state; // exposition only

        template<class Sch>
            class backend_for; // exposition only
    public:
        using scheduler_concept = scheduler_t;

        template<class Sch, class Allocator = allocator<void>>
            requires (!same_as<task_scheduler, remove_cvref_t<Sch>>)
            && scheduler<Sch>
        explicit task_scheduler(Sch&& sch, Allocator alloc = {});
    };
}
```

```

ts-sender see below schedule();

template <class Sndr, class Env>      // exposition only
    see below bulk-transform(Sndr&& sndr, const Env& env);

    friend bool operator==(const task_scheduler& lhs, const
        task_scheduler& rhs) noexcept;

template<class Sch>
    requires (!same_as<task_scheduler, Sch>) && scheduler<Sch>
    friend bool operator==(const task_scheduler& lhs, const
        Sch& rhs) noexcept;

private:
    shared_ptr<voidparallel_scheduler_backend> sch_; // exposi-
        tion only                                     // see
        [exec.sysctxrepl.psb]
};
}

```

1. task_scheduler is a class that models scheduler (33.6 [exec.sched]). Given an object s of type task_scheduler, let *SCHED*(s) be the sched member of the object owned by s.sch_.
2. For an lvalue r of type derived from receiver_proxy, let *WRAP-RCVR*(r) be an object of a type that models receiver and whose completion handlers result in invoking the corresponding completion handlers of r.

```

template<class Sch>
struct backend-for : parallel_scheduler_backend {
    // exposition only
    explicit backend-for(Sch sch) :
        sched_(std::move(sch)) {}

    void schedule(receiver_proxy& r, span<byte> s)
        noexcept override;
    void schedule_bulk_chunked(size_t shape,
        bulk_item_receiver_proxy& r,
        span<byte> s) noexcept
        override;
    void schedule_bulk_unchunked(size_t shape,
        bulk_item_receiver_proxy& r,
        span<byte> s) noexcept
        override;

    Sch sched_; // exposition only
};

```

3. Let sndr be a sender whose only value completion signature is set_value_t() and for which the expression get_completion_scheduler<set_value_t>(get_env(sndr)) == sched_ is true.

```
void schedule(receiver_proxy& r, span<byte> s) noexcept
    override;
```

4. *Effects*: Constructs an operation state *os* with `connect(schedule(sched_), WRAP-RCVR(r))` and calls `start(os)`.

```
void schedule_bulk_chunked(size_t shape,
    bulk_item_receiver_proxy& r,
    span<byte> s) noexcept
    override;
```

5. *Effects*: Let *chunk_size* be an integer less than or equal to *shape*, let *num_chunks* be $(\text{shape} + \text{chunk_size} - 1) / \text{chunk_size}$, and let *fn* be a function object such that for an integer *i*, *fn*(*i*) calls `r.execute(i * chunk_size, m)`, where *m* is the lesser of $(i + 1) * \text{chunk_size}$ and *shape*. Constructs an operation state *os* as if with `connect(bulk(sndr, par, num_chunks, fn), WRAP-RCVR(r))` and calls `start(os)`.

```
void schedule_bulk_unchunked(size_t shape,
    bulk_item_receiver_proxy& r,
    span<byte> s) noexcept
    override;
```

6. *Effects*: Let *fn* be a function object such that for an integer *i*, *fn*(*i*) is equivalent to `r.execute(i, i + 1)`. Constructs an operation state *os* as if with `connect(bulk(sndr, par, shape, fn), WRAP-RCVR(r))` and calls `start(os)`.

```
template<class Sch, class Allocator =
    allocator<void>>
    requires(!same_as<task_scheduler, remove_cvref_t<Sch>>) && scheduler<Sch>
explicit task_scheduler(Sch&& sch, Allocator alloc =
    {});
```

2. *Effects*: Initialize *sch_* with `allocate_shared<backend-for-remove_cvref_t<Sch>>>(alloc, std::forward<Sch>(sch))`.

[Editor's note: Paragraphs 3-7 are kept unmodified. Remove paragraphs 8-12 and add the following paragraphs:]

see below `schedule()`;

8. *Returns*: a prvalue *sndr* whose type *Sndr* models sender such that:

- (8.1) — `get_completion_scheduler<set_value_t>(get_env(sndr))` is equal to `*this`.
- (8.2) — If a receiver *rcvr* is connected to *sndr* and the resulting operation state is started, calls `sch_>schedule(r, s)`, where

- (8.2.1) — `r` is a proxy for `rcvr` with base `system_context_replaceability::receiver_proxy` (33.15 [exec.par.scheduler]) and
- (8.2.2) — `s` is a preallocated backend storage for `r`.

```
template <class BulkSndr, class Env>    // exposition only
    see below bulk_transform(BulkSndr&& bulk_sndr, const Env&
                             env);
```

9. *Constraints*: `sender_in<BulkSndr, Env>` is true and either *sender-for*<BulkSndr, bulk_chunked_t> or *sender-for*<BulkSndr, bulk_unchunked_t> is true.

10. *Returns*: a prvalue `sndr` whose type models sender such that:

- (10.1) — `get_completion_scheduler<set_value_t>(get_env(sndr))` is equal to `*this`.
- (10.2) — `bulk_sndr` is connected to an unspecified receiver if a receiver `rcvr` is connected to `sndr`. If the resulting operation state is started,
 - (10.2.1) — If `bulk_sndr` completes with values `vals`, let `args` be a pack of lvalue subexpressions designating objects decay-copied from `vals`. Then
 - (10.2.1.1) — If `bulk_sndr` is the result of calling `bulk_chunked(child, policy, shape, f)`, `sch->schedule_bulk_chunked(shape, r, s)` is called where `r` is a bulk chunked proxy for `rcvr` with callable `f` and arguments `args`, and `s` is a preallocated backend storage for `r`.
 - (10.2.1.2) — Otherwise, `bulk_sndr` is the result of calling `bulk_unchunked(child, policy, shape, f)`. Calls `sch->schedule_bulk_unchunked(shape, r, s)` where `r` is a bulk unchunked proxy for `rcvr` with callable `f` and arguments `args`, and `s` is a preallocated backend storage for `r`.
 - (10.2.2) — All other completion operations are forwarded unchanged.

[Editor's note: In 33.15 [exec.par.scheduler], add a new paragraph after paragraph 3, another before paragraph 10, and change paragraphs 10 and 11 as follows:]

3. The expression `get_forward_progress_guarantee(sch)` returns `forward_progress_guarantee::parallel`.

?. The expression `get_completion_scheduler<set_value_t>(get_env(schedule(sch))) == sch` evaluates to true.

...as before ...

?. Let `sch` be a subexpression of type `parallel_scheduler`. For subexpressions `sndr` and `env`, if `tag_of_t<Sndr>` is neither `bulk_chunked_t` nor `bulk_unchunked_t`, the expression `sch.bulk-transform(sndr, env)` is ill-formed; otherwise, let `child`, `pol`, `shape`, and `f` be subexpressions equal to the arguments used to create `sndr`.

10. ~~`parallel_scheduler` provides a customized implementation of the `bulk_chunked` algorithm (33.9.12.11 [exec.bulk]). If a receiver `rcvr` is connected to the sender returned by `bulk_chunked(sndr, pol, shape, f)`~~ When the tag type of `sndr` is `bulk_chunked_t`, the expression `sch.bulk-transform(sndr, env)` returns a sender such that if it is connected to a receiver `rcvr` and the resulting operation state is started, then:

(10.1) — If ~~`sndr`~~`child` completes with values `vals`, let `args` be a pack of lvalue subexpressions designating `vals`, then `b.schedule_bulk_chunked(shape, r, s)` is called, where

(10.1.1) — `r` is a bulk chunked proxy for `rcvr` with callable `f` and arguments `args` and

(10.1.2) — `s` is a preallocated backend storage for `r`.

(10.2) — All other completion operations are forwarded unchanged.

[*Note*: Customizing the behavior of `bulk_chunked` affects the ~~default~~ implementation of `bulk`. — *end note*]

11. ~~`parallel_scheduler` provides a customized implementation of the `bulk_unchunked` algorithm (33.9.12.11 [exec.bulk]). If a receiver `rcvr` is connected to the sender returned by `bulk_unchunked(sndr, pol, shape, f)`~~ When the tag type of `sndr` is `bulk_unchunked_t`, the expression `sch.bulk-transform(sndr, env)` returns a sender such that if it is connected to a receiver `rcvr` and the resulting operation state is started, then:

(11.1) — If ~~`sndr`~~`child` completes with values `vals`, let `args` be a pack of lvalue subexpressions designating `vals`, then `b.schedule_bulk_unchunked(shape, r, s)` is called, where

(11.1.1) — `r` is a bulk unchunked proxy for `rcvr` with callable `f` and arguments `args` and

(11.1.2) — `s` is a preallocated backend storage for `r`.

(11.2) — All other completion operations are forwarded unchanged.

§ 9 Appendix A: The planned fix

Our willingness to remove algorithm customization depends on our confidence that we can add it back later without breaking code. Section [Restoring algorithm customization in C++29](#) talks about how we would go about this. This appendix fleshes out some of the details.

§ 9.1 Mission statement

A sender expression represents a task graph, the nodes of which are asynchronous operations. Every async operation is started on some execution context, the *starting context*, and completes on another execution context, the *completing context*. The two might be the same, but that’s irrelevant.

Note:

This is a simplification. Some senders like `when_all` can complete on one of several contexts. We solve that problem with domains as described below.

Imagine we assign each execution resource a color. The mission then is to paint every node in the task graph with the colors of its starting and completing contexts. Once we know where each operation will start and complete, we can use that information to pick the right algorithm implementation.

In regards to customization, each color can be thought to represent not an individual execution resource, but rather a set of algorithm implementations. Two different execution resources might use the same set of algorithm implementations, so they would have the same “color”. In fact, most execution resources will use the default set of algorithm implementations, in which case they all have the same color.

That’s not always the case though. A thread pool would not want to use the default implementation of `bulk` for example – that would be serial. The thread pool would have a different color corresponding to its set of preferred algorithm implementations.

In `std::execution` today, this notion of color is called a “domain”. A domain is a tag type that is used to select a set of algorithm implementations. Schedulers, which are stand-ins for execution resources, advertize their domain with the `get_domain` query.

§ 9.2 Achieving the mission

Completing the mission requires two things:

1. Identifying the starting and completing domain of every operation in the task graph, and
2. Using that information to select the preferred implementation for the algorithm that operation represents.

Let's take these two separately.

§ 9.2.1 Coloring the graph

§ 9.2.1.1 Early Customization

So-called “early” customization, which determines the return type of `then(sndr, fn)` for example, is predicated on the fact that senders know the domain on which they will complete. As discussed above, that's false. Many senders only know where they will complete once they know where they will start, which isn't known until the sender is connected to a receiver.

So early customization is irreparably broken. There is no plan to add it back.

§ 9.2.1.2 Late Customization

That leaves late customization, which is performed by the connect customization point. The receiver, which is an extension of caller, knows where the operation will start. *If the sender is given this information – that is, if the sender is told where it will start – it can accurately report where it will complete.* This is the key insight.

When connect queries a sender's attributes for its domain, it should pass the receiver's environment. That way a sender has all the information available when computing its completion domain.

§ 9.2.1.3 `get_completion_domain`

It is sometimes the case that a sender's value and error completions can happen on different domains. For example, imagine trying to schedule work on a GPU. If it succeeds, you are in the GPU domain, and Bob's your uncle. If scheduling fails, however, the error cannot be reported on the GPU because we failed to make it there!

So asking a sender for a singular completion domain is not flexible enough. We have three separate queries for a sender's completion scheduler: `get_completion_scheduler<set_[value|error|stopped]_t>`. Similarly, we should have three separate queries for a sender's completion domain: `get_completion_domain<set_[value|error|stopped]_t>`.

Note:

If we have the `get_completion_scheduler` queries, why do we need `get_completion_domain`? We can ask the completion scheduler for its domain, right? The answer is that a sender like `when_all(s1, s2)` doesn't know what scheduler it will complete on. It completes on the context of whichever sender, `s1` or `s2`, finishes last. But if `s1` and `s2` have the same completion *domain*, it doesn't matter that we do not know the completion scheduler. The *domain* determines the preferred set of algorithm implementations. Hence we

need separate queries for the completion domain. (Additionally, when_all must require that all of its child senders share a common domain.)

The addition of the completion domain queries creates a nice symmetry as shown in the table below (with additions in green):

	Receiver	Sender
Query for scheduler	get_scheduler	get_completion_scheduler<set_value_t> get_completion_scheduler<set_error_t> get_completion_scheduler<set_stopped_t>
Query for domain	get_domain	get_completion_domain<set_value_t> get_completion_domain<set_error_t> get_completion_domain<set_stopped_t>

For a sender `sndr` and an environment `env`, we can get the sender's completion domain as follows:

```
auto completion_domain = get_completion_domain<set_value_t>
    (get_env(sndr), env);
```

A sender like `just()` would implement this query as follows:

```
template <class... Values>
class just_sender {
private:
    struct attrs {
        template <class Env>
        auto query(get_completion_domain_t<set_value_t>, const Env&
            env) const noexcept {
            // just(...) completes where it starts. the domain of the
            // environment is where
            // the sender will start, so return that.
            return get_domain(env);
        }
        //...
    };

public:
    attrs get_env() const noexcept {
        return attrs{};
    }

    //...
};
```

Note:

A query that accepts an additional argument is novel in `std::execution`, but the query system was designed to support this usage. See 33.2.2 [\[exec.-queryable.concept\]](#).

With the addition of the `get_completion_domain<...>` queries that can accept the receiver's environment, `connect` can now “paint” the operation with its starting and completing colors, aka domains. When passed arguments `snr` and `rcvr`, the starting domain is:

```
// Get the operation's starting domain:
auto starting_domain = get_domain(get_env(rcvr));
```

To get the completion domain (when the operation completes successfully):

```
// Get the operation's completion domain for the value channel:
auto completion_domain = get_completion_domain<set_value_t>
    (get_env(snr), get_env(rcvr));
```

Now `connect` has all the information it needs to select the correct algorithm implementation. Great!

But this presents the `connect` function with a dilemma: how does it use *two* domains to pick *one* algorithm implementation?

Consider that the starting domain might want a say in how `start` works, and the completing domain might want a say in how `set_value` works. So should we let the starting domain customize `start` and the completing domain customize `set_value`?

No. `start` and `set_value` are bookends around an async operation; they must match. Often `set_value` needs state that is set up in `start`. Customizing the two independently is madness.

§ 9.2.2.1 Solving the double-dispatch problem

Note:

The following is more speculative than what has been described so far.

A possible solution I have been exploring is to bring back sender transforms. Each domain can apply its transform in turn. I do not yet have reason to believe the order matters, but it is important that when asked to transform a sender, a domain knows whether it is the “starting” domain or the “completing” domain.

Here is how a domain might customize bulk when it is the completing domain:

```
struct thread_pool_domain {
    template <sender_for<bulk_t> Snr, class Env>
        auto transform_sender(set_value_t, Snr&& snr, const Env&
                               env) const {
        //...
    }
};
```

Since it has `set_value_t` as its first argument, this transform is only applied when `thread_pool_domain` is an operation's completion domain. Had the first argument been

start_t, the transform would only be used when thread_pool_domain is a starting domain.

transform_sender

In this reimagined customization design, the connect CPO does a few things:

1. Determines the starting and completing domains,
2. Applies the completing domain's transform (if any),
3. Applies the starting domain's transform (if any) to the resulting sender,
4. Connects the twice-transformed sender to the receiver.

The first three steps are doing something different than connecting a sender and receiver, so it makes sense to factor them out into their own utility. I call it transform_sender here, but it does not need to be normative since only connect will call it.

The new transform_sender looks like this:

```
template <class Domain, class Tag, class Sndr, class Env>
concept has_sender_transform_for = requires (Sndr(*make_sndr()),
    const Env env) {
    Domain().transform_sender(Tag(), make_sndr(), env);
}

template <class Domain, class Tag>
constexpr auto transform_sender_recurse = overload_set{
    []<class Self, class Sndr, class Env>(this Self self, Sndr&&
        sndr, const Env& env)
        -> decltype(auto) requires has_sender_transform_for<Domain,
            Tag, Sndr, Env> {
            return self(Domain().transform_sender(Tag(), std::for-
                ward<Sndr>(sndr), env));
        },
    []<class Sndr, class Env>(Sndr&& sndr, const _Env&) -> Sndr {
        return std::forward<Sndr>(sndr);
    }
};

template <class Sndr, class Env>
auto transform_sender(Sndr&& sndr, const Env& env) {
    auto starting_domain      = get_domain(env);
    auto completing_domain    = get_completion_domain<set_value_t>
        (get_env(sndr), env);

    auto starting_transform    = transform_sender_recurse<de-
        cltype(starting_domain), start_t>;
    auto completing_transform  = transform_sender_recurse<de-
        cltype(completing_domain), set_value_t>;

    return starting_transform(completing_transform(std::for-
        ward<Sndr>(sndr), env), env);
}
```

With this definition of `transform_sender`, `connect(sndr, rcvr)` is equivalent to `transform_sender(sndr, get_env(rcvr)).connect(rcvr)`.

§ 9.2.2.2 Revisiting the problematic example

Let's see how this new approach addresses the problems noted in the motivating example [above](#). The troublesome code is:

```
namespace ex = std::execution;
auto sndr = ex::starts_on(gpu, ex::just()) | ex::then(fn);
std::this_thread::sync_wait(std::move(sndr));
```

The problem with [P3718](#) describes how the current design and the “fixed” one proposed in [\[P3718R0\]](#) go off the rails while determining the domain in which the function `fn` will execute, causing it to use a CPU implementation instead of a GPU one.

In the new design, when the `then` sender is being connected to `sync_wait`'s receiver, the starting domain will still be the `default_domain`, but when asking the sender where it will complete, the answer will be different. Let's see how:

- When asked for its completion domain, the `then` sender will ask the `starts_on` sender where it will complete, as if by:

```
auto&& tmp1 = ex::starts_on(gpu, ex::just());
auto dom1 = ex::get_completion_domain<ex::set_value_t>
(ex::get_env(tmp1), ex::get_env(rcvr));
```

- In turn, the `starts_on` sender asks the `just()` sender where it will complete, *telling it where it will start*. (This is the new bit.) It looks like:

```
auto&& tmp2 = ex::just();
// ask for the gpu scheduler's domain:
auto gpu-dom = ex::get_completion_domain<ex::set_value_t>
(gpu);
// construct an env that reflects the fact that tmp2 will be
// started on the gpu:
auto env2 = ex::env{ex::prop{ex::get_scheduler, gpu},
                   ex::prop{ex::get_domain, gpu-dom},
                   ex::get_env(rcvr)};
// pass the new env when asking `just()` for its completion
// domain:
auto dom2 = ex::get_completion_domain<ex::set_value_t>
(ex::get_env(tmp2), env2);
```

- The `just()` sender, when asked where it will complete, will respond with the domain on which it is started. That information is provided by the `env2` environment passed to the query: `get_domain(env2)`. That will return `gpu-dom`.
- Having correctly determined that the `then` sender will start on the default domain and complete on the GPU domain, `connect` can select the right implementation for the `then` algorithm. It does that by calling:

```
return ex::transform_sender(sndr, ex::get_env(rcvr)).connect(rcvr);
```

The `transform_sender` call will execute the following (simplified):

```
ex::default_domain().transform_sender(ex::start,
                                       gpu-dom.transform_sender(ex::set_value, sndr, ex::get_env(rcvr)),
                                       ex::get_env(rcvr))
```

The `default_domain` does not apply any transformation to then senders, so this expression reduces to:

```
gpu-dom.transform_sender(ex::set_value, sndr, ex::get_env(rcvr))
```

So, in the new customization scheme, the GPU domain gets a crack at transforming the then sender before it is connected to a receiver, as it should.

§ 10 References

[P3325R1] Eric Niebler. 2024-07-14. A Utility for Creating Execution Environments.

<https://wg21.link/p3325r1>

[P3718R0] Eric Niebler. 2025-06-28. Fixing Lazy Sender Algorithm Customization, Again.

<https://wg21.link/p3718r0>